



Daric ReRAM Access Control Application Notes

Revision	0.3
Status	Draft
Function	CROSSBAR, Inc.
Path and Filename	Daric_ReRAM_Access_Control.doc
File Type	Microsoft Word

Table of Contents

1	DARIC RERAM ACCESS CONTROL	5
1.1	RERAM INTRODUCTION	5
1.1.1	IFR (INFORMATION ARRAY)	7
1.1.2	DATA SLOT REGION MAP (64KB)	9
1.1.3	KEY SLOT REGION MAP (64KB)	10
1.2	RERAM READ MECHANISM	12
1.3	RERAM WRITE MECHANISM.....	13
1.4	RERAM ACCESS CONTROL PATHS.....	14
1.4.1	RERAM ACCESS CONTROL MECHANISM	15
1.4.2	RERAM SIZE CONFIGURATION	16
1.4.3	INSTRUCTION FETCH TO CODE REGION	18
1.4.4	DATA ACCESS CONTROL TO CODE REGION	18
1.4.5	DATA ACCESS CONTROL TO ONE-WAY COUNTER REGION	19
1.4.6	DATA ACCESS CONTROL TO ACCESS CONTROL REGION.....	19
1.4.7	DATA ACCESS CONTROL TO DATA SLOT REGION	23
1.4.8	KEY ACCESS CONTROL TO KEY SLOT REGION	23
1.4.9	DATA ACCESS CONTROL TO IFR REGION.....	24
1.4.10	DATA ACCESS CONTROL IN OEM MODE	25
1.5	RRC REGISTERS	26
1.5.1	RRCCR.....	27
1.5.2	RRCFD	27
1.5.3	RRCSR	28
1.5.4	RRCFR	28
1.5.5	RRCSET	29
1.5.6	RRCRST	29
1.5.7	RRCRD	30
1.5.8	RRCAR	30
1.6	RERAM OPERATION REFERENCE FLOW	30
1.6.1	REGULAR RERAM WRITE OPERATION FLOW.....	30
1.6.2	SPECIAL RERAM ONE-WAY COUNTER OPERATION FLOW	31
1.6.3	SPECIAL RERAM PROG-ONLY OPERATION FLOW.....	31
1.6.4	SPECIAL RERAM ACCESS CONTROL REGION UPDATE FLOW	31
1.6.5	HIGH-LEVEL SECURE CERTIFICATE OPERATION FLOW.....	31
2	DESIGN REVIEW INFORMATION	38

Figure 1-1 Block Diagram.....错误!未定义书签。

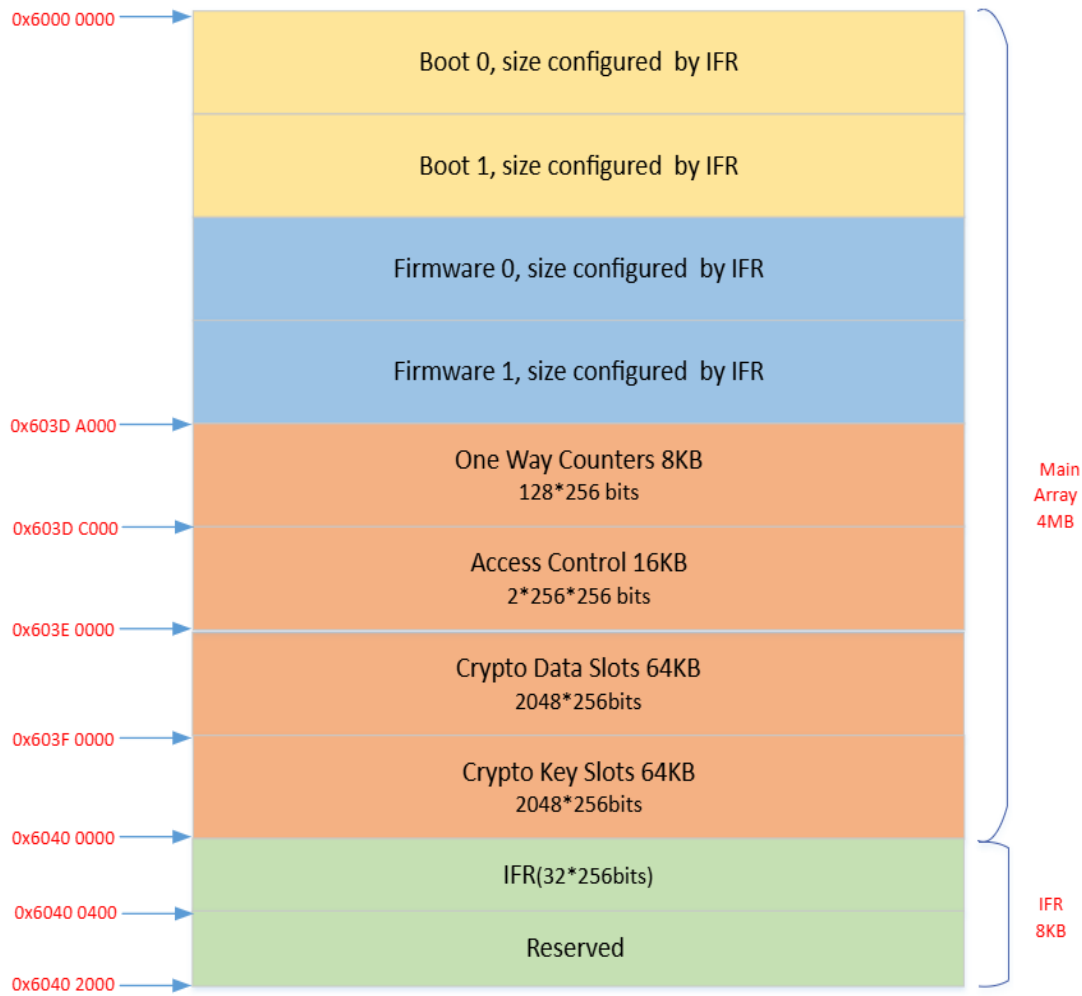
Table of Tables

Table 12-1 Actions Items 38

1 *Daric ReRAM Access Control*

1.1 *ReRAM Introduction*

The DARIC features up to 4Mbyte of embedded ReRAM memory available for storing programs and data. The ReRAM memory contains two blocks (IP0 and IP1) of TSMC ReRAM devices with 128bit bus width each, which forms up 256bit bus width in total. ReRAM memory visible to SW can be divided into two regions: main array and IFR array. ***ReRAM default value will be zero to users if not initialized in the CP flow.***



ReRAM Memory Map

The memories outside of ReRAM can be used as code, which are TCM/SRAM, will be recognized as FW1.

So for all the access control, FW1 should be limited with very minimal permissions.

In most cases, set FW1 size to zero in IFR is suggested.

There are more breakdown regions of ReRAM memory maps to serve different use cases:

- **Code Region**

- located at 0x6000_0000 to 0x603D_9FFF max (3,944KB)
- max 3 regions, boot0/boot1/fw0
- min 1 user only
- configurable address region
- support single write (256bit) and page write (32 sets of 256bit)

- **One-way Counters**

- located at 0x603D_A000 to 0x603D_BFFF (8KB)
- only available under user mode
- no user limitation and write only region
- software controls the writing times, no more than 10K write times/each word, 256-bit
- support single write only

- **Access Control Region**

- located at 0x603D_C000 to 0x603D_FFFF (16KB)
- only available under user mode
- including access control configuration of key slots, and data slots
- support single write only to update access control configuration

- **Crypto Data Slots 2048 sets**

- located at 0x603E_0000 to 0x603E_FFFF (64KB)
- only available under user mode
- support program only or write operation
- can provide A user-writeable, set-only memory slots
- support single and page write

- **Crypto Key Slots 2048 sets**

- located at 0x603F_0000 to 0x603F_FFFF (64KB)
- only available under user mode
- 255 sets of authentication link root key
- support single and page write

- **IFR (Information Array)**

- located at 0x6040_0000 to 0x6040_03FF (1KB)
- configurable under user mode for lab testing debug, but frozen for mass production and downloaded only by test mode
- life cycle pattern
- chip system data: trimming, user mapping assignment, S/N
- read/write disable configuration for each key slot/data slot/code block
- support single write only

1.1.1 IFR (Information Array)

NO	bits	content	amba address	detailed trimmings	privileged user mode access control (by AMBA bus)
0	256*1	life cycle pattern	0x6040_0000	life cycle pattern flag, default setting: user pattern	R/W locked by default
1	256*1	analog trimmings -0	0x6040_0020	analog IP Trimmings Group 0	R/W locked by default
2	256*1	analog trimmings -1	0x6040_0040	analog IP Trimmings Group 1	R/W locked by default
3	256*1	analog trimmings -2	0x6040_0060	analog IP Trimmings Group 2	R/W locked by default
4	256*1	system control -0	0x6040_0080	cfg_info, default setting: "daric_cfg_code" IP1: 0x0000_0000_0000_0000_0000_0000_0000_0000 IP0: 0x0000_0000_0000_0000_0000_da11_ccf9_c0de	R/W locked by default
5	256*1	system control -1	0x6040_00A0	rev	...
6	256*1	system control -2	0x6040_00C0	cfg_rrsub_size, default setting: whole RRAM open to any boot/app code amba address[31:0] = {10'b0110_0000_00, IP0[x+7: x], 14'h0} unit block size: 16KB IP1: 0x0000_0000_0000_0000_0000_0000_0000_0000 IP0: 0x00ff_0000_ffff_ffff_0000_ffff_ffff_0000 IP0[127:120]: reram start IP0[119:112]: reram end IP0[111:104]: m7 init (IV) IP0[103:96]: boot0 start IP0[95:88]: boot1 start IP0[87:80]: fw0 start IP0[79:72]: fw1 start IP0[71:64]: fw1 end IP0[63:56]: rv init IP0[55:48]: boot0 start IP0[47:40]: boot1 start IP0[39:32]: fw0 start IP0[31:24]: fw1 start IP0[23:16]: fw1 end IP0[15:8]: sramcut (sram0+1, separately 4bit each) IP0[7:5]: user_trustkey_enable, [5]: boot1, [6], fw0, [7], fw1 IP0[4:3]: Reserved IP0[2]: RV default mm IP0[1:0]: rv default user {0:boot0, 1:boot1, 2:fw0, 3:fw1}	In privilege mode only. the same IP1 settings can disable Read/Write accesses.

7	256*1	system control -3	0x6040_00E0	rev	...
8	256*1	system control -4	0x6040_0100	rev	...
9	256*1	system control -5	0x6040_0120	rev	...
10	256*1	sce nvr -6	0x6040_0140	rev	...
11	256*1	system control -7	0x6040_0160	rev	...
12	256*1	system control -8	0x6040_0180	rev	R/W locked by default ...
13	256*1	system control -9	0x6040_01A0	rev	...
14	256*1	system control -10	0x6040_01C0	rev	...
15	256*1	system control -11	0x6040_01E0	rev	...
16	256*1	system control -12	0x6040_0200	rev	...
17	256*1	system control -13	0x6040_0220	rev	...
18	256*1	system control -14	0x6040_0240	rev	...
19	256*1	system control -15	0x6040_0260	rev	...

20	256*1	access control -0	0x6040_0280	cfg_rrsub_rw_boot0, default setting: whole boot0 open to read/write operation IP0[127:120]: {test_write_dis_byte_boot0} IP0[119:112]: {test_read_dis_byte_boot0} IP0[111:104]: user_code_cfg_boot0	In privilege mode only. the same IP1 settings can disable Read/Write accesses.
21	256*1	access control -1	0x6040_02A0	cfg_rrsub_rw_boot1, default setting: whole boot1 open to test read/write operation IP0[127:120]: {test_write_dis_byte_boot1} IP0[119:112]: {test_read_dis_byte_boot1} IP0[111:104]: user_code_cfg_boot1	In privilege mode only. the same IP1 settings can disable Read/Write accesses.
22	256*1	access control -2	0x6040_02C0	cfg_rrsub_rw_fw0, default setting: whole fw0 open to test read/write operation IP0[127:120]: {test_write_dis_byte_fw0} IP0[119:112]: {test_read_dis_byte_fw0} IP0[111:104]: user_code_cfg_fw0	In privilege mode only. the same IP1 settings can disable Read/Write accesses.
23	256*1	access control -3	0x6040_02E0	cfg_rrsub_rw_fw1, default setting: whole fw1 open to test read/write operation IP0[127:120]: {test_write_dis_byte_fw1} IP0[119:112]: {test_read_dis_byte_fw1} IP0[111:104]: user_code_cfg_fw1	In privilege mode only. the same IP1 settings can disable Read/Write accesses.
24~31	256*8	access control – 4~11	0x6040_0300 ~ 0x6040_03E0	rev	R/W locked in usermode

1.1.2 Data Slot Region Map (64KB)

data slot	size	core address
data slot 0	256 x 1	0x603E_0000 - 603E_001C
data slot 1	256 x 1	0x603E_0020 - 603E_003C
.....		
.....		

data slot 2046	256 x 1	0x603E_FFC0 - 603E_FFDC
data slot 2047	256 x 1	0x603E_FFE0 - 603E_FFFC

1.1.3 Key Slot Region Map (64KB)

key slot	size	core address	link-key
Auth key slot 0	256 x 1	0x603F_0000 - 603F_001C	don't care trustkey/akey used as root access key
Auth key slot 1	256 x 1	0x603F_0020 - 603F_003C	don't care trustkey/akey used as root access secret
Auth key slot 2	256 x 1	0x603F_0040 - 603F_005C	key_cfg used as boot1 access key
Auth key slot 3	256 x 1	0x603F_0060 - 603F_007C	key_cfg used as boot1 access secret
Auth key slot 4	256 x 1	0x603F_0080 - 603F_009C	key_cfg used as fw0 access key
Auth key slot 5	256 x 1	0x603F_00A0 - 603F_00BC	key_cfg used as fw0 access secret
Auth key slot 6	256 x 1	0x603F_00C0 - 603F_00DC	key_cfg used as fw1 access key
Auth key slot 7	256 x 1	0x603F_00E0 - 603F_00FC	key_cfg used as fw1 access secret
....			
....			
Auth key slot 254	256 x 1	0x603F_0FC0 - 603F_09DC	key_cfg
Auth key slot 255	256 x 1	0x603F_0FC0 - 603F_09FC	don't care trustkey/akey
General key slot 256	256 x 1	0x603F_1000 - 0x603F_101C	key_cfg
General key slot 257	256 x 1	0x603F_1020 - 0x603F_103C	key_cfg
....	...	...	
General key slot 2047	256 x 1	0x603F_FFE0 - 0x603F_FFFC	key_cfg

There are two kinds of different key slots.

- Auth Key Pair (Akey)**

- Even number key slots are designed to store keys for HMAC.
- Odd number key slots are designed to store secrets for HMAC to compare with the calculated results.
- If the HMAC result is compared correctly, this auth key pair is set to trust state.
- **Special Auth Key slot 255**

- *Auth key slot 255: the SW can write any value or keep it as zero. This key slot doesn't care about any restriction and usually provide the certificate pass result.*
- *If any general key slot or auth key slot, and its akey_id is 255, this key slot is not required HMAC certificate to be used.*

• General Key (Gkey)

- The general key can be used separately, not in pairs as Auth key.
- Each general key should have its own auth key ID. It means before accessing this general key, the related auth key must pass the HMAC certificate flow. Otherwise, access is not allowed.
- Use the odd number auth key slot number as the auth key ID.

More details can be found in **chapter** Data access control to Data Slot region

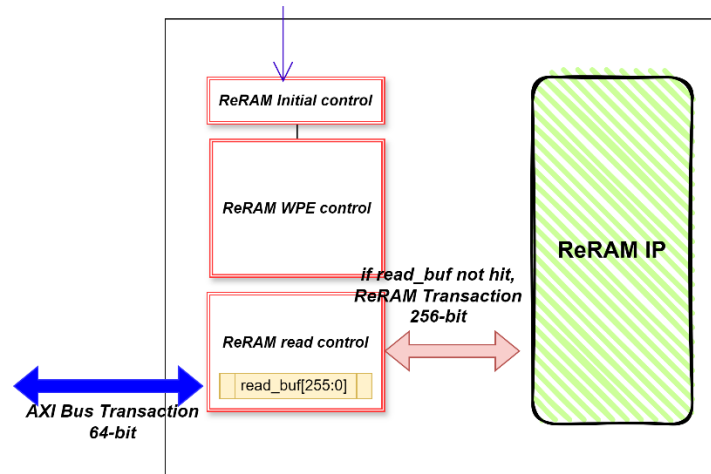
The data slot region can be accessed by different masters. Usually, it requires higher level user mode to access the data slot for crypto application.

master	user mode	slot owner	segid	rd_dis	wr_dis	access
x.Read	x	NO_OWNER, SHARED	x	0	x	allow
x.Write	x	NO_OWNER, SHARED	x	x	0	allow
CM7.CORE. Read	privilege	= cm7_coreuser	x	0	x	allow
CM7.CORE. Write	privilege	= cm7_coreuser	x	x	0	allow
RV. CORE.Read	machine	= rv_coreuser	x	0	x	allow
RV. CORE.Write	machine	= rv_coreuser	x	x	0	allow
SCE.Read	exclusive/secure	= sce_user	match	0	x	allow
SCE.Write	exclusive/secure	= sce_user	match	x	0	allow

Any other operation will be denied.

Key access control to Key Slot Region

1.2 ReRAM read mechanism



ReRAM read flow diagram

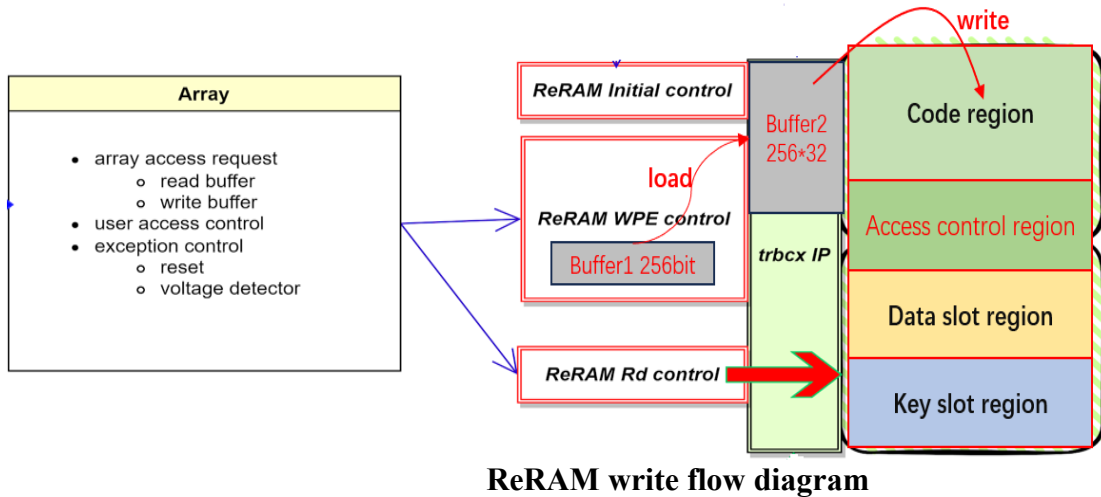
ReRAM can support 256-bit read out data each transaction. The DARIC supports single read buffer design to maximize the instruction fetch performance. The AXI bus transaction is based on 64-bit data width.

If the target read data is already perfected in the read_buf[255:0], then the ReRAM controller will directly read the data from the read_buf and no wait delays for these kinds of read operation.

If the target read data is not hit in the read_buf, the ReRAM controller will read the data from ReRAM IP array and store the data into the read_buf. At the same time, the controller will send back the data through AXI bus to the CPU cores.

Assume one ReRAM array transaction can provide 4 sets of continuous 64-bit instructions. Then the average read time is $(6 * \text{Period}_{\text{ramclk}} + 3 * \text{Period}_{\text{aclk}})$, which includes one 256-bit ReRAM read access time for the first unhit 64-bit, and the following three hit 64-bit AXI bus transaction time.

1.3 ReRAM write mechanism



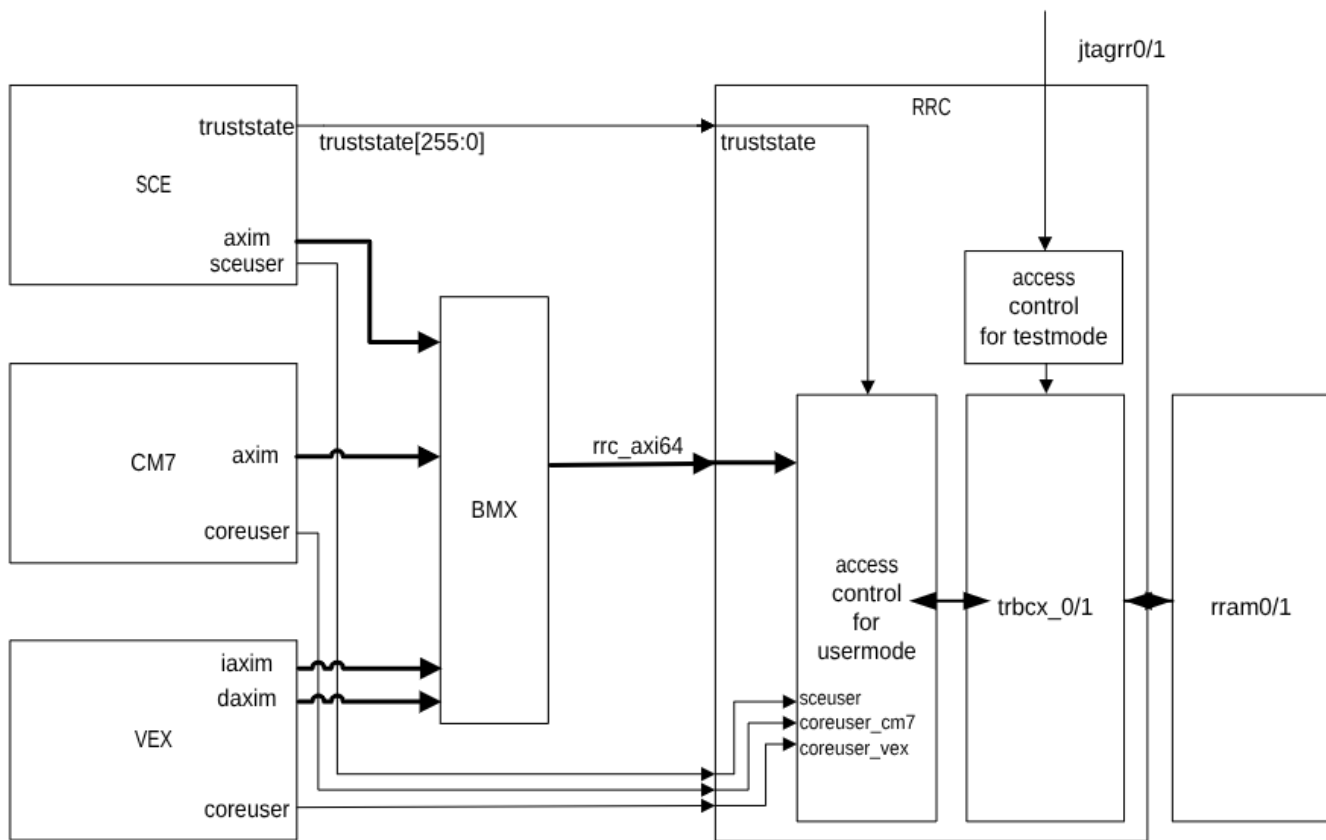
ReRAM can support multiple word write operations. This process includes three steps: pre-load, load and write.

- **1st pre-load step**
The CPU cores access ReRAM by AXI-64bit bus. The pre-load means to use CPU write transaction to generate **one 256-bit write data Buffer1** for ReRAM, which may split into several bus transactions, such as 8 times 32-bit CPU write, or 4 times 64-bit CPU write.
- **2nd load step**
After pre-load step, the ReRAM IP requires **the Buffer1 data** to be loaded into **internal Buffer2** of trbcx module. The Buffer2 can store multiple N sets of 256-bit data(N=1~32).
- **3rd write step**
After the load step, all the write data are ready to be written into real ReRAM array. This step needs high voltage and long process time.

The ReRAM writing time is based on the differences between original stored data and new write data. The average writing time is as below:

- **from All 0 to All 1**
 - AVG: 1-word-write time is ~50us
32-word-write time is ~800us.
 - MAX: 1-word-write time is <100us
32-word-write time is <1200us.
- **from All 1 to All 0 (worst pattern)**
 - AVG: 1-word-write time is ~120us
32-word-write time is ~1500us.
 - MAX: 1-word-write time is <200us
32-word-write time is <2560us.

1.4 ReRAM access control paths



ReRAM access control path diagram

From the diagram above, the DARIC provides three paths to access the ReRAM memory. (1) The first one is that the CPU cores (CM7 core/RISC-V core) can access ReRAM, (2) the second one is that the SCE DMA master can access ReRAM through AXI bus, (3) the third one is the access path through JTAG port, enabled in test mode, for lab debug and silicon production purpose.

When CPU accesses the ReRAM, it will also provide the core_user ID information (carried by AXI side channel) to the ReRAM controller as the user who is to access ReRAM. The ReRAM controller will match the user with its access configuration to check if this user can access the target slave ReRAM address region.

The SDMA master uses a similar method. The SDMA will also provide the sce_user ID information (carried by AXI side channel) to the ReRAM controller as the user who is using SDMA to access ReRAM.

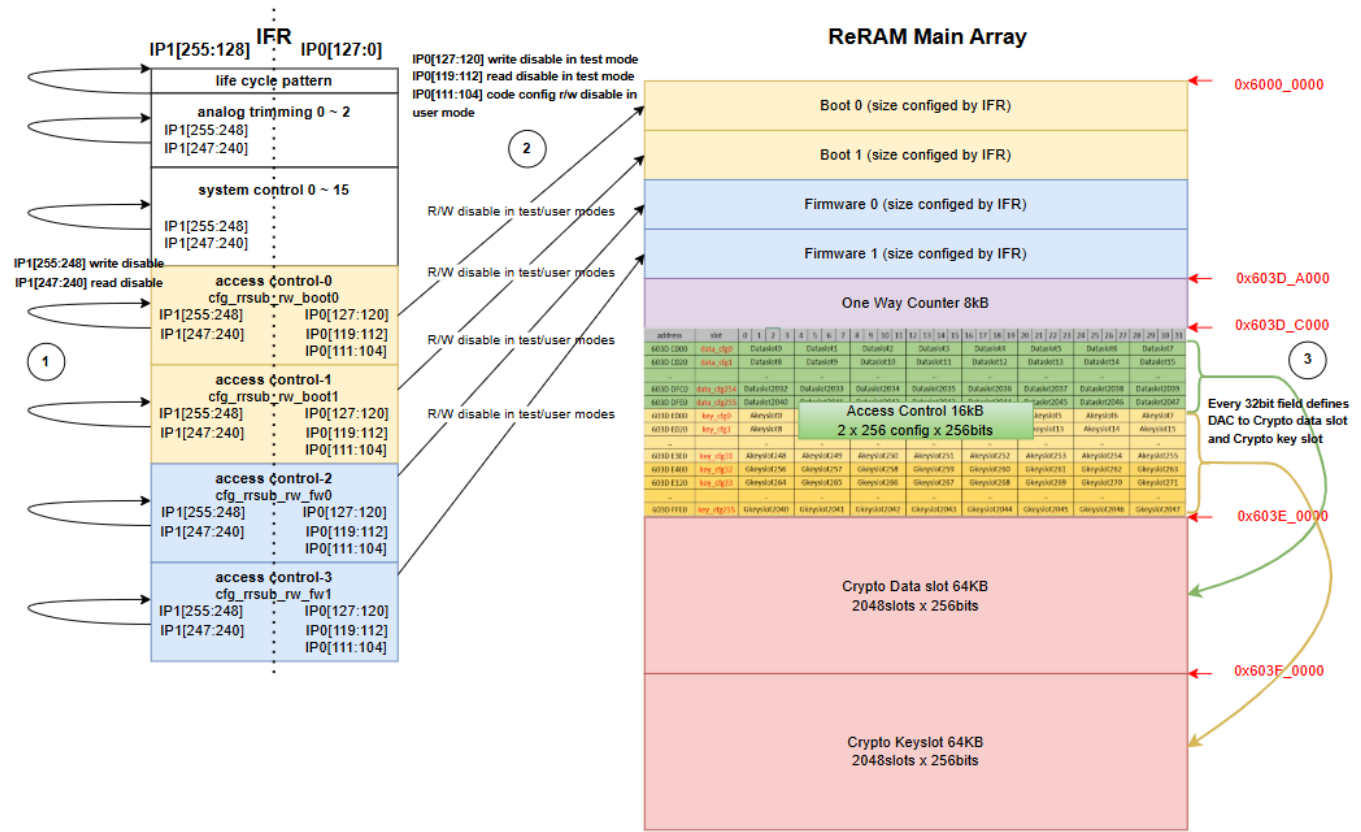
Access control includes code execution, data read, and data write operations. There are different access control restrictions based on target operations.

All these access control functions can be enabled separately in the RRCCR register. If any errors happen, the interruption will be generated, and the specific flag bit will be set in RRCFR register.

If enabled the NMI function(RRCCR[15]), the error flag will generate both NMI and normal interrupt. More details can be referred to the *chapter*

RRC Registers.

1.4.1 ReRAM Access Control Mechanism



ReRAM Access Control Diagram

RRAM	target		ac cfg		user mode	oem mode
IFR	each IFR item (256b)	1	IFR.IP1[255:248]	w	by pattern match, for all users	jtag
	each IFR item (256b)	1	IFR.IP1[247:240]	r	by pattern match, for all users	jtag
MA	each code region	2	IFR.ac0~3.IP0[127:120]	w	-	jtag
		2	IFR.ac0~3.IP0[119:112]	r	-	jtag
		2	IFR.ac0~3.IP0[111:104]	r/w	each user has a bit to control	-
	dataslot	3	datacfg in ac region	r/w/sce	coreuser/sceuser/segid match	-
	keyslot	3	keycfg in ac region	r/w/sce	coreuser/sceuser/segid/ts match	-

ReRAM Access Control Mechanism is to control the Read/Write permission of IFR and Main Array. There are three control mechanisms:

- to control the access permission of IFR itself. The IFR contains 32 entries with 256bits each. The access of each entry can be individually programmed by writing '0x3a' (PM_WRITE_DIS)

to IP1[255:248] to disable Write permission and writing ‘0x96’ (PM_READ_DIS) to disable Read permission.

This work both in OEM mode and user mode.

2. to control the access permission to Boot0, Boot1, Firmware0, Firmware1, Access control regions of Main Array. Their permissions were handled by ‘access control-0 ~ access control-3’ of IFR.
 - a. ‘access control-0~3’ IP0[127:120], IP0[119:112] are the access control patterns for the 4 user regions in OEM mode only.

Taking ‘access control-0’ as an example, setting ‘0x3a’ (PM_WRITE_DIS) to IP0[127:120] can disable Write permission of Boot0, and ‘0x96’ (PM_READ_DIS) can disable Read permission of Boot0.
 - b. ‘access control-0~3’ IP0[111:104] is used to control the access permission of different users (Boot0/Boot1/FW0/FW1) in user mode.
 - c. Access Control Region was composed of 256 data configuration slots and 256 key configuration slots, their permissions were controlled by IFR’s ‘access control-4 ~ access control-7’ and ‘access control-9 ~ access control-11’, respectively.
3. to control the access permission of Crypto Data Slot and Crypto Key Slot regions in Main Array. They are handled by Access Control Region. For details, please refer to the section of ‘Data access to Access Control Region’.

1.4.2 ReRAM size configuration

The ReRAM memory region configuration is located at IFR 0x6040_00C0.

The detailed bit definition is as below:

- **ReRAM array size settings**
 - ✓ bit[127:120]: ReRAM start
 - ✓ bit [119:112]: ReRAM end
 - ✓ default value: 0x00ff, which means the whole ReRAM is used.
 The ReRAM start address = {10'b0110_0000_00, bit[127:120], 14'h0}
 The ReRAM last address = {10'b0110_0000_00, bit[119:112], 14'h3FFF}
- **CM7 user region settings**
 - ✓ bit [111:104]: CM7 init (IV)
 - ✓ bit [103:96]: boot0 start
 - ✓ bit [95:88]: boot1 start
 - ✓ bit [87:80]: fw0 start
 - ✓ bit [79:72]: fw1 start
 - ✓ bit [71:64]: fw1 end
 - ✓ default value: 0x0000_ffff_ffff, which means ONLY the boot0 is used.
 The boot0 start address = {10'b0110_0000_00, bit[103:96], 14'h0}
 The boot0 last address = {10'b0110_0000_00, bit[95:88], 14'h3FFF}
 - ✓ The CM7 init address should be the same as the SW power-on start address(usually it's

boot0 start address or boot1 start address)

- ✓ *To restrict fw1, here fw1 start/end should be set same number, to 'compress' fw1 space into zero.*

■ **RISC-V user region settings**

- ✓ *bit [63:56]: rv init*
- ✓ *bit [55:48]: boot0 start*
- ✓ *bit [47:40]: boot1 start*
- ✓ *bit [39:32]: fw0 start*
- ✓ *bit [31:24]: fw1 start*
- ✓ *bit [23:16]: fw1 end*
- ✓ *Similar to CM7's bit definition.*
- ✓ *Usage suggestions:*
 - *single core: RISC-V core only, this value can be set to any value based on application requirements.*
 - **multiple cores:**
 1. *CM7 settings will be used instead of these settings, set bit[63:16] the same as bit[111:64].*
 2. *SW design should treat the RISC-V core to be set as one user, and this user cannot be used by CM7 core, sinc these two cores instruction systems are not the same and compatible. RISC-V should set default user(bit[1:0]) to match SW design.*
 3. *If both CM7 and RISC-V use the same user, the ReRAM controller cannot identify their differences and will treat them as the same user. At any time, only one core can access ReRAM.*
- ✓ *To restrict fw1, here fw1 start/end should be set same number, to 'compress' fw1 space into zero.*

■ **SRAM array size settings**

- ✓ *bit[15:12]: SRAM1 size reduction settings*
 - *bit[7], size cut enable*
 - *bit[6:4], size range*
 - *default, 0x0, disable and no size reduction.*
 - *if enabled, the SRAM1 valid address range < {bit[6:4], 15'h0}*
- ✓ *bit[11:8]: SRAM0 size reduction settings*
 - *bit[3], size cut enable*
 - *bit[2:0], size range*
 - *default, 0x0, disable and no size reduction.*
 - *if enabled, the SRAM0 valid address range < {bit[2:0], 15'h0}*

■ **Other user-related settings**

- ✓ *bit [7:5] : user_trustkey_enable*
 - *bit[5]: user_trustkey_enable of boot1 user*
 - *bit[6]: user_trustkey_enable of fw0 user*
 - *bit[7]]: user_trustkey_enable of fw1 user*
 - *default value: 0x0, disable high-level security code certificate.*
- ✓ *bit [2]: RISC-V core default mm, machine mode setting*
- ✓ *bit [1:0]: RISC-V core default user*
 - *0:boot0, 1:boot1, 2:fw0, 3:fw1*

■ **Reserved bits:**

- ✓ *bit[255:128]: Reserved for CP testing use only. Always write 0.*
- ✓ *bit[15:8]: Reserved. Always write 0.*

- ✓ bit [4:3]: *Reserved. Always write 0.*

1.4.3 Instruction fetch to Code Region

The DARIC supports two different kinds of code execution access control.

- **default setting:** similar to general MCU, no restriction for all the users to run the code instruction.
- **high-level secure bootup sequence:** required basic HMAC certificate pass before boot1, fw0 and fw1 code execution. The detailed reference flow for high-level security certificate can be found in *chapter* High-level secure certificate operation flow.

1.4.4 Data access control to Code Region

Two rules were applied to the control of Code Region. Rule#1 must be compliant all the time. If Rule#2 is invoked, then it must comply with both Rule#1 & Rule#2 together.

1.4.4.1 Rule #1 IFR IP0[111:104] user_code_cfg

The ReRAM data access control to code region configuration is located in IFR. Each user has its own configuration settings.

- **boot0 configuration**

- ✓ located at IFR 0x6040_0280.
- ✓ bit [111:104]: user_code_cfg_boot0, to assign different users' read and write access to boot0 region. Default value =0, means always allowed.
If the bit is set to 1, means disabled, the access is not allowed.

<i>operation</i>	<i>boot0</i>	<i>boot1</i>	<i>fw0</i>	<i>fw1</i>
read boot0	Y	bit[106]	bit[108]	bit[110]=0
write boot0	Y	bit[107]	bit[109]	bit[111]=0

- ✓ Other bits reserved for CP testing ONLY. *Always write 0.*
- ✓ Tie bit[111:110] zero to restrict fw1 access boot0.

- **boot1 configuration**

- ✓ located at IFR 0x6040_02A0.
- ✓ bit [111:104]: user_code_cfg_boot1, to assign different users' read and write access to boot1 region. Default value =0, means always allowed.
If the bit is set to 1, means disabled, the access is not allowed.

<i>operation</i>	<i>boot0</i>	<i>boot1</i>	<i>fw0</i>	<i>fw1</i>
read boot1	bit[104]	Y	bit[108]	bit[110]=0
write boot1	bit[105]	Y	bit[109]	bit[111]=0

- ✓ Other bits reserved for CP testing ONLY. *Always write 0.*
- ✓ Tie bit[111:110] zero to restrict fw1 access boot1.

- **fw0 configuration**

- ✓ located at IFR 0x6040_02C0.
- ✓ bit [111:104]: user_code_cfg_fw0, to assign different users' read and write access to fw0 region. Default value =0, means always allowed.
If the bit is set to 1, means disabled, the access is not allowed.

<i>operation</i>	<i>boot0</i>	<i>boot1</i>	<i>fw0</i>	<i>fw1</i>
read fw0	bit[104]	bit[106]	Y	bit[110]=0
write fw0	bit[105]	bit[107]	Y	bit[111]=0

- ✓ Other bits reserved for CP testing ONLY. *Always write 0.*
- ✓ Tie bit[111:110] zero to restrict fw1 access fw0.

▪ fw1 configuration

- ✓ located at IFR 0x6040_02E0.
- ✓ bit [111:104]: user_code_cfg_fw1, to assign different users' read and write access to fw1 region. Default value =0, means always allowed.
If the bit is set to 1, means disabled, the access is not allowed.

<i>operation</i>	<i>boot0</i>	<i>boot1</i>	<i>fw0</i>	<i>fw1</i>
read fw1	bit[104]	bit[106]	bit[108]	Y
write fw1	bit[105]	bit[107]	bit[109]	Y

- ✓ Other bits reserved for CP testing ONLY. *Always write 0.*

1.4.4.2 Rule #2, trust state

Bits in IFR.cfg_rrsub[7:5]:

IP0[7:5] : user_trustkey_enable, [5]: boot1, [6]: fw0, [7]: none

are to enable/disable the trust state requirement of opening the code user's region.

Boot0 is accessible by default.

- *user_trustkey_enable[5]* for boot1, the required truststate is [3];
- *user_trustkey_enable[6]* for fw0, the required truststate is [5];

When the required truststate is not set, the user region is not accessible, including

1.4.5 Data access control to One-way Counter Region

Similar to general MCU, there is no restriction for all the users to use the one-way counter region. Each one-way counter should be written less than 10K times.

1.4.6 Data access control to Access Control Region

To enable to data access control to access control region, all 7 rules must be met.

rule 2#	boot0/boot1 only (cm7/vex)
rule 3#	If cm7 is enabled, it must be privilege mode
rule 4#	If vex is enabled, it must be machine mode
rule 5#	sce CAN NOT rd/wr

rule 6#	data op only (rd/wr, no execution)
---------	--------------------------------------

This region can only be accessed by the privilege user (boot0 and boot1). It's used to store access control configurations of all the data slots and key slots. Each data slot or key slot should have its own 32-bit access control configuration bits.

address	slot	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	
603D C000	data_cfg0	Dataslot0			Dataslot1			Dataslot2			Dataslot3			Dataslot4			Dataslot5			Dataslot6			Dataslot7											
603D C020	data_cfg1	Dataslot8			Dataslot9			Dataslot10			Dataslot11			Dataslot12			Dataslot13			Dataslot14			Dataslot15											
..		..			..			..			..			..			..			..			..											
603D DFC0	data_cfg254	Dataslot2032			Dataslot2033			Dataslot2034			Dataslot2035			Dataslot2036			Dataslot2037			Dataslot2038			Dataslot2039											
603D DFE0	data_cfg255	Dataslot2040			Dataslot2041			Dataslot2042			Dataslot2043			Dataslot2044			Dataslot2045			Dataslot2046			Dataslot2047											
603D E000	key_cfg0	Akeyslot0			Akeyslot1			Akeyslot2			Akeyslot3			Akeyslot4			Akeyslot5			Akeyslot6			Akeyslot7											
603D E020	key_cfg1	Akeyslot8			Akeyslot9			Akeyslot10			Akeyslot11			Akeyslot12			Akeyslot13			Akeyslot14			Akeyslot15											
..		..			..			..			..			..			..			..			..											
603D E3E0	key_cfg31	Akeyslot248			Akeyslot249			Akeyslot250			Akeyslot251			Akeyslot252			Akeyslot253			Akeyslot254			Akeyslot255											
603D E400	key_cfg32	Gkeyslot256			Gkeyslot257			Gkeyslot258			Gkeyslot259			Gkeyslot260			Gkeyslot261			Gkeyslot262			Gkeyslot263											
603D E120	key_cfg33	Gkeyslot264			Gkeyslot265			Gkeyslot266			Gkeyslot267			Gkeyslot268			Gkeyslot269			Gkeyslot270			Gkeyslot271											
..		..			..			..			..			..			..			..			..											
603D FFE0	key_cfg255	Gkeyslot2040			Gkeyslot2041			Gkeyslot2042			Gkeyslot2043			Gkeyslot2044			Gkeyslot2045			Gkeyslot2046			Gkeyslot2047											

▪ Data slot control region (green)

- Item: Each Item controls access to the corresponding slot in the Data Slots region, refer to Dataslot control configuration definition.
- Size: 2048*4=8KB
- Address: 0x603D C000-0x603D DFFF

• Key slot control region (orange)

- Item: Each Item controls access to the corresponding slot in the Key Slots region, refer to Keyslot control configuration definition
- Size: 2048*4=8KB
- Address: 0x603D E000-0x603D FFFF
- Akey: Authentication link root key, it controls Gkey access

• data_cfg slot (256-bit)

- Each data_cfg slot includes 8 data slots access control configuration items.
- Each data slot has 32-bits of its own access control configuration.
- Assume data_cfg slot N, its bit assignment is as below:
 - o bit[31:0] -> data slot (0+N*8) access control configuration
 - o bit[63:32] -> data slot (1+N*8)
 - o bit[95:64] -> data slot (2+N*8)
 - o bit[127:96] -> data slot (3+N*8)
 - o bit[159:128] -> data slot (4+N*8)
 - o bit[191:160] -> data slot (5+N*8)
 - o bit[223:192] -> data slot (6+N*8)
 - o bit[255:224] -> data slot (7+N*8)

• key_cfg slot (256-bit)

- Each key_cfg slot includes 8 key slots access control configuration items.

- **Each key slot** has 32-bits of its own access control configuration.
- **Assume key _cfg slot N, its bit assignment is as below:**
 - bit[31:0] -> key slot (0+N*8) access control configuration
 - bit[63:32] -> key slot (1+N*8)
 - bit[95:64] -> key slot (2+N*8)
 - bit[127:96] -> key slot (3+N*8)
 - bit[159:128] -> key slot (4+N*8)
 - bit[191:160] -> key slot (5+N*8)
 - bit[223:192] -> key slot (6+N*8)
 - bit[255:224] -> key slot (6+N*8)

- Key slot access control configuration word (32-bit)**

keycfg(32-bit)		
[0]	core_rd_dis	core read disable, default = 0, enabled
[1]	core_wr_dis	core write disable, default = 0, enabled
[2]	sce_rd_dis	sce read disable, default = 0, enabled
[3]	sce_wr_dis	sce write disable, default = 0, enabled
[7:4]	--	Reserved
[15:8]	seg_id[7:0]	<i>sce algo ID, should be set to match SW design.</i> <i>if HMAC certificate: set 8'h1f</i> <i>other cases set to match SCE registers' definition:</i> <i>SCE GDMA : XCHCR_SEGID, 0x4002_901C</i> <i>SCE SDMA : SCHCR_SEGID, 0x4002_903C</i> <i>These registers are defined by SW.</i>
[23:16]	slot_owner[7:0]	[7]: fw1 [6]: fw0 [5]: boot1 [4]: boot0 [3:0]: Reserved
[31:24]	akey_id[7:0]	Auth Key ID (if linked to any auth key, just set the target auth key id number : 0-255)

- Data slot access control configuration word (32-bit)**

datacfg(32-bit)		
[0]	core_rd_dis	core read disable, default = 0, enabled
[1]	core_wr_dis	core write disable, default = 0, enabled
[2]	sce_rd_dis	SCE read disable, default = 0, enabled
[3]	sce_wr_dis	SCE write disable, default = 0, enabled
[7:4]	--	Reserved
[15:8]	seg_id[7:0]	<i>sce algo ID, should be set to match SW design.</i> <i>if HMAC certificate: set 8'h1f</i> <i>other cases set to match SCE registers' definition:</i> <i>SCE GDMA : XCHCR_SEGID, 0x4002_901C</i> <i>SCE SDMA : SCHCR_SEGID, 0x4002_903C</i> <i>These registers are defined by SW.</i>
[23:16]	slot_owner[7:0]	[7]: fw1 [6]: fw0 [5]: boot1 [4]: boot0 [3:0]: Reserved
[24]	write mode	=0, write (0->1, 1->0, both) =1, program-only (0->1 only) <i>This bit is the configuration only, how to invoke this ReRAM operation to data slot region, please refer to the chapter ReRAM operation reference flow</i>
[31:25]	others	Reserved

1.4.7 Data access control to Data Slot region

The data slot region can be accessed by different masters. Usually, it requires higher level user mode to access the data slot for crypto application.

master	user mode	slot owner	segid	rd_dis	wr_dis	access
x.Read	x	NO_OWNER, SHARED	x	0	x	allow
x.Write	x	NO_OWNER, SHARED	x	x	0	allow
CM7.CORE. Read	privilege	= cm7_coreuser	x	0	x	allow
CM7.CORE. Write	privilege	= cm7_coreuser	x	x	0	allow
RV. CORE.Read	machine	= rv_coreuser	x	0	x	allow
RV. CORE.Write	machine	= rv_coreuser	x	x	0	allow
SCE.Read	exclusive/secure	= sce_user	match	0	x	allow
SCE.Write	exclusive/secure	= sce_user	match	x	0	allow

Any other operation will be denied.

1.4.8 Key access control to Key Slot Region

The key slot region can be accessed by different masters. Usually, it requires higher level user mode to access the data slot for crypto application.

master	user mode	slot owner	segid	rd_dis	wr_dis	access
x.Read	x	NO_OWNER, SHARED	x	0	x	allow
x.Write	x	NO_OWNER, SHARED	x	x	0	allow
CM7.CORE. Read	privilege	= cm7_coreuser	x	0	x	allow
CM7.CORE. Write	privilege	= cm7_coreuser	x	x	0	allow
RV. CORE.Read	machine	= rv_coreuser	x	0	x	allow
RV. CORE.Write	machine	= rv_coreuser	x	x	0	allow
SCE.Read	secure	= sce_user	match	0	x	allow
SCE.Write	secure	= sce_user	match	x	0	allow

Any other operation will be denied.

Besides the special user mode, it also requires the auth key trust state set to 1. This means, before accessing the target key slot, the operator must pass the related auth key pair HMAC certificate.

master	user mode	slot owner	segid	rd_dis	wr_dis	akey trust	access
x.Read	x	NO_OWNER, SHARED	x	0	x	x	allow
x.Write	x	NO_OWNER, SHARED	x	x	0	x	allow
CM7.CORE. Read	privilege	= cm7_coreuser	x	0	x	1	allow
CM7.CORE. Write	privilege	= cm7_coreuser	x	x	0	1	allow
RV. CORE.Read	machine	= rv_coreuser	x	0	x	1	allow

RV.CORE.Write	machine	= rv_coreuser	x	x	0	1	allow
SCE.Read	secure	= sce_user	match	0	x	1	allow
SCE.Write	secure	= sce_user	match	x	0	1	allow

Any other operation will be denied.

To be more specific, the rules of the access control are:

keyslot	dataslot		
v	v	rule 1#	coreuser == keycfg/datacfg.userid ----> un-encoded vector, cant be shared for multi-user
v	v	rule 2#	cm7 must be privilege mode vex must be machine mode
v	v	rule 3#	keycfg/datacfg.core rd/wr disable
v	v	rule 4#	sce must be in sec_op
v	v	rule 5#	keycfg.sce rd/wr disable bits
v	v	rule 6#	sce keytype(segid) match
v	--	rule 7#	linked key passed (truststate)
v	v	rule 8#	data op only
v	v	rule 9#	reram user only

1.4.9 Data access control to IFR region

The IFR region is used to store the system level configuration. 9 rules were applied to access control. They all must be met.

rule 1#	[255:248][247:240] r/w seal
rule 2#	cm7 must be privilege mode
rule 3#	vex must be machine mode
rule 4#	sce CAN NOT rd/wr
rule 8#	data op
rule 9#	reram user only

But before mass production, by skipping the disabled write access step, then the DARIC can support SW to access the IFR region during the lab testing or sample testing process. ***This region can be accessed by the privilege user (boot0 and boot1) for debug and development process only.***

The SW doesn't need to do the access control. The final version of settings should be downloaded and then disabled the write access datapath under test mode or OEM mode during the CP or FT process. The users cannot modify the IFR settings for mass production.

The detailed CP or FT testing steps can be referred to in the document Daric NTO Test Mode User Guide.

1.4.10 Data access control in OEM Mode

All access control described above occurs in user mode.

During the production process, to enable step-by-step program downloads in test mode:

- Download our boot0 in the CP.
- Download the user's own boot1 in the user's FT. At this point, we want the tester can't read out or modify the boot0.
- If the product will further download firmware0 in the next user level, we also want tester to be unable to access boot0/boot1.

Therefore, in test mode, we implement OEM mode by adding user access control:

- OEM mode is not a dedicated mode, but rather a combination of test mode and access control.
- The implementation path of OEM mode is completely different from that of user mode. In test mode, the ReRAM is exposed to the outside world via JTAG, so all access control is achieved by the address and operation rule checks on the JTAG path and blocking illegal access.

OEM mode achieves irreversible lifecycle advancement through a series of control bit changes.

RRAM	target		ac cfg		oem mode
IFR	each IFR item (256b)	1	IFR.IP1[255:248]	w	jtag
	each IFR item (256b)	1	IFR.IP1[247:240]	r	jtag
MA	each code region	2	IFR.ac0~3.IP0[127:120]	w	jtag
		2	IFR.ac0~3.IP0[119:112]	r	jtag

1.5 RRC Registers

Addresses	0x4000 0000					
Register Name	Offset	Size	Type	Access	Default	Description
RRCCR	0x0000	32	CR	R/W	0x00000000	ReRAM control register
RRCFD	0x0004	32	CR	R/W	0x00000007	ReRAM clock configuration
<i>RRCSR</i>	<i>0x0008</i>	<i>32</i>	<i>SR</i>	<i>R</i>	<i>0x00000000</i>	<i>ReRAM debug only</i>
RRCFR	0x000C	32	FR	R	0x00000000	ReRAM access control error flag register
<i>RRCSRSET0</i>	<i>0x0014</i>	<i>32</i>	<i>SR</i>	<i>R</i>	<i>0x00000000</i>	<i>ReRAM debug only,[31:0]</i>
<i>RRCSRSET1</i>	<i>0x0018</i>	<i>32</i>	<i>SR</i>	<i>R</i>	<i>0x00000000</i>	<i>ReRAM debug only,[63:32]</i>
<i>RRCSRST0</i>	<i>0x001C</i>	<i>32</i>	<i>SR</i>	<i>R</i>	<i>0x00000000</i>	<i>ReRAM debug only,[31:0]</i>
<i>RRCSRST1</i>	<i>0x0020</i>	<i>32</i>	<i>SR</i>	<i>R</i>	<i>0x00000000</i>	<i>ReRAM debug only,[63:32]</i>
<i>RRCSRSD0</i>	<i>0x0024</i>	<i>32</i>	<i>SR</i>	<i>R</i>	<i>0x00000000</i>	<i>ReRAM debug only,[31:0]</i>
<i>RRCSRSD1</i>	<i>0x0028</i>	<i>32</i>	<i>SR</i>	<i>R</i>	<i>0x00000000</i>	<i>ReRAM debug only,[63:32]</i>
<i>RRCAR</i>	<i>0x00F0</i>	<i>32</i>	<i>AR</i>	<i>W</i>	<i>0x00000000</i>	<i>ReRAM suicide AR, limited use only</i>

1.5.1 RRCCR

- Description: ReRAM control register
- Address Offset: 0x0000
- Reset Value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
evc_trigger_enable															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

- Bit[0], low power mode configuration of ReRAM IP
 - 0: nap
 - 1: power down
- Bit[1], access mode configuration of ReRAM IP
 - 0: data mode for write
 - 1: command mode for write
- Bit[11:2], reserved.
- Bit[12], *code_access_err CR bit*
It's 'b0 after boot up. Software write over RRCCR will turn it to 'b1. And no way to turn back to 'b0.
Software turn this on ASAP is strongly suggested.
- Bit[14:13], reserved.
- Bit[15], *rrc nmi enable CR bit*
 - 0: *disable ReRAM controller trigger access error NMI to system*
 - 1: *enable ReRAM controller trigger access error NMI to system*
- Bit[31:16], *event trigger one-way counter enable CR bits*
 - [16]: =0, disable
=1, enable for axi_one_way_counter 240, 0x603D_BFC0, 256-bit ReRAM word
 -
 - [31]: =0, disable
=1, enable enable for axi_one_way_counter 255, 0x603D_BFFC

1.5.2 RRCFD

- Description: ReRAM clock configuration register
- Address Offset: 0x0004
- Reset Value: 0x0000 0007

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
reserved											fd[4:0]				

- Bit[4:0]: ReRAM IP working frequency rramclk configuration
 - rramclk should be typical 50MHz, max 100MHz.
 - SW can change this value based on different system clock clktop frequency to make ReRAM running at fastest speed if needed.
 - $\text{frequency}_{\text{rramclk}} = \text{frequency}_{\text{clktop}} / (\text{fd} + 1)$
 - here are some examples:
when pll disabled, clktop = 32MHz (osc), then rramclk = 4MHz

when pll enabled, clktop = 400MHz (pll), then rramclk = 50MHz

1.5.3 RRCSR

- Description: ReRAM *internal trbcx IP status probe register (debug only)*
- Address Offset: 0x0008
- Reset Value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
reserved						internal trbcx IP status									

- Bit [0]: trc_busy:
 - 0: idle
 - 1: busy
- Bit[4:1:] trc_ip_cmd[3:0] definition
 - 4'h0: idle
 - 4'h1: read
 - 4'h2: load
 - 4'h3: write (HV operation)
 - other: reserved
- Bit[5]: trc_info_lock_err
 - 0: no info region lock error
 - 1: errors happen
- Bit[6]: trc_err
 - 0: no trc function error
 - 1: errors happen
- Bit[9:7]: ecc_err, ecc algo error numbers (2C2D ECC supported)
 - 3'b000: error free
 - 3'b001: 1-bit error
 - 3'b010: 2-bit error
 - 3'b100: reserved
- If no exception happens, these register bits will stay 0.

1.5.4 RRCFR

- Description: ReRAM access control error flag register
- Address Offset: 0x000C
- Reset Value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
reserved															

- Bit[0]: key_access_err flag
 - 0: no errors happen
 - 1: invalid key slot access happens, error interrupt
- Bit[1]: data_access_err flag
 - 0: no errors happen
 - 1: invalid data slot access happens, error interrupt
- Bit[2]: code_access_err flag

- 0: no errors happen
- 1: invalid code region access happens, error interrupt
- Bit[3]: cfg_access_err flag
 - 0: no errors happen
 - 1: invalid access control cfg region access happens, error interrupt
- Bit[4]: ifr_access_err flag
 - 0: no errors happen
 - 1: invalid IFR region access happens, error interrupt
- ***All these bits can only be cleared by SW write "1" to specific bit.***
- ***The RCCFR status is for auxiliary reference only and does not always reflect actual hardware intercepts due to potential reporting gaps. The core Access Control logic is unaffected and remains reliable even if the flag is not set.***

1.5.5 RRCSET

- Description: ReRAM SET probe register (debug only)
- Address Offset: 0x0014
- Reset Value: 0x0000 0000

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
47	46	45	44	43	42	41	40	39	38	37	36	35	34	33	32
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

- *probe trc_set_failure[63:0], program 1 failure bit counts for page operation.*
- *If no exception happens, these register bits will stay 0.*

1.5.6 RRCRST

- Description: ReRAM SET probe register (debug only)
- Address Offset: 0x001C
- Reset Value: 0x0000 0000

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
47	46	45	44	43	42	41	40	39	38	37	36	35	34	33	32
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

- *probe trc_reset_failure[63:0], erase 0 failure bit counts for page operation.*
- *If no exception happens, these register bits will stay 0.*

1.5.7 RRCRD

- Description: ReRAM SET probe register (debug only)
- Address Offset: 0x0024
- Reset Value: 0x0000 0000

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
47	46	45	44	43	42	41	40	39	38	37	36	35	34	33	32
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

- probe *trc_fourth_read_failure*[63:0], write 0/1 failure bit counts for page operation.
- If no exception happens, these register bits will stay 0.

1.5.8 RRCAR

- Description: ReRAM AR register
- Address Offset: 0x00F0
- Reset Value: 0x0000 0000

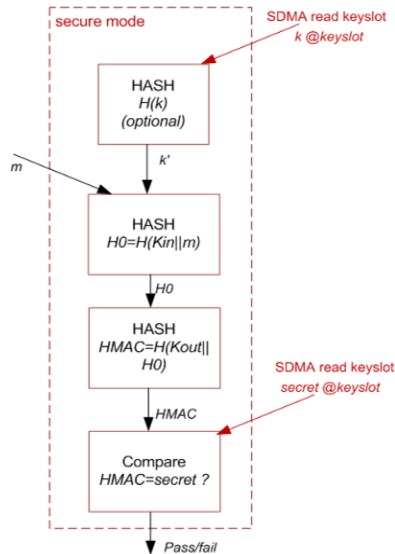
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
set value															

- Bit[31:0]: write PM_RRAM_SUICIDE (0x0000_2468) value to initial a suicide flow. Erase all the ReRAM.
- The AR register always read out all zeros.
- This register is limited used only. Not required in every product application. The product application security level will decide if provided this function to external users.

1.6 ReRAM operation reference flow

1.6.1 Regular ReRAM write operation flow

- (1) Set RRCCR[1] =0 (power-on default value), data mode for input write data
- (2) write 256-bit to internal write buffer1, Support 8 times 32-bit write, or 4 times 64-bit write to ReRAM address
- (3) Set RRCCR[1] =1, command mode for LOAD or WRITE command
- (4) Write LOAD command(0x5200) to ReRAM address.
- (5) Support Single Write or Page Write. Support 1 to 32 times of 256-bit loading to the same ReRAM Page.
- (6) For multiple 256-bit load, just repeat step 2/3/4 for all the target data and address.
- (7) Write WRITE command(0x9528) to ReRAM address
- (8) Read back the ReRAM address, you can check if the write is successful.



- HMAC_check = (HMAC_sign (m, @k) == @secret)
 - Normal input: m(Message)
 - Keyslot read: k, secret
 - Output: HMAC_check (pass or fail)
- **Suggested configuration:**
 - Data slot inputs can be read or written in by CPU or SDMA.
 - Auth key slot can only be read or written by SDMA.
 - General key slot can be read or written by CPU or SDMA.

The SW can define what kind of the security level is required based on the real application. **The following example is to describe the whole flow of HASH algo calculation with high-level secure key certificate function.**

Step 1: Configure Reram data slot security level

```
uint32 t_reram_tsnode_data_config[] = { 0x00100100, 0x00100100, 0x00100100, 0x00100100, 0x00100100, 0x00100100, 0x00100100, 0x00100100 };
arraycheck_reram("\n Reram Truststate data configuring.....", 0x603dc000, reram_tsnode_data_config, 0);
//arraycheckread("reram tsnode data config. reading from 0x603dc000", 0x603dc000, 1);
```

we set [15:8] of 0x603dc000 to 0x603dc01c to "01" as the key of HMAC256 will be stored in key slot 1.

Step 2: Configure Reram key slot security level

```
uint32 t_reram_tsnode_key_config[] = { 0x00100100, 0x00101f00, 0x001100100, 0x001101f00, 0x001100100, 0x001101f00, 0x001100100, 0x001101f00 };
arraycheck_reram("\n Reram Truststate key configuring.....", 0x603de000, reram_tsnode_key_config, 0);
//arraycheckread("reram tsnode key config. reading from 0x603de000", 0x603de000, 1);
```

Write 0x603D_E000 0x00100100 (keyslot0)
Write 0x603D_E004 0x00101f00 (keyslot1)

If the first key HMAC certificate passes, the controller will allow the valid user to access the target auth key slot 2 and auth key slot 3.

Write 0x603D_E008 0x01100100 (keyslot2, akey=1)
Write 0x603D_E00C 0x01101f00 (keyslot3, akey=1)

If the second key HMAC certificate passes, the controller will allow the valid user to access the target auth key slot 4 and auth key slot 5.

Write 0x603D_E010 0x03100100 (keyslot4, akey=3)
Write 0x603D_E014 0x03101f00 (keyslot5, akey=3)

If the third key HMAC certificate passes, the controller will allow the valid user to access the auth target key slot 6 and auth key slot 7.

Write 0x603D_E018 0x05100100 (keyslot6, akey=5)
Write 0x603D_E01C 0x05101f00 (keyslot7, akey=5)

Even number key slots are designed to store keys, Odd number key slots are designed to store secrets for HMAC to compare with the calculated results.

- segid "01" represent to store key.
- segid "1f" represent to store secret.

Step 3: Hash parameter initialization

```

/*****Hash parameter initialization*****/
singlecheck("GLB_suben ", 0x40028004, 0xb); // clk enable

uint32_t sha256[] = {
0x6A09E667, 0xBB67AE85, 0x3C6EF372, 0xA54FF53A,
0x510E527F, 0x9B05688C, 0x1F83D9AB, 0x5BE0CD19,
0x428A2F98, 0x71374491, 0xB5C0FBCF, 0xE9B5DBA5,
0x3956C25B, 0x59F111F1, 0x923F82A4, 0xAB1C5ED5,
0xD807AA98, 0x12835B01, 0x243185BE, 0x550C7DC3,
0x72BE5074, 0x80DEB1FE, 0x9BDC06A7, 0xC19BF174,
0xE49B69C1, 0xEFBE4786, 0x0FC19DC6, 0x240CA1CC,
0x2DE92C6F, 0x4A7484AA, 0x5CB8A9DC, 0x76F988DA,
0x983E5152, 0xA831C66D, 0xB00327C8, 0xBF597FC7,
0xC6E00BF3, 0xD5A79147, 0x06CA6351, 0x14292967,
0x27B70A85, 0x2E1B2138, 0x4D2C6DFC, 0x53380D13,
0x650A7354, 0x766A0ABB, 0x81C2C92E, 0x92722C85,
0xA28FE8A1, 0xA81A664B, 0xC24B8B70, 0xC76C51A3,
0xD192E819, 0xD6990624, 0xF40E3585, 0x106AA070,
0x19A4C116, 0x1E376C08, 0x2748774C, 0x34808C85,
0x391C0CB3, 0x4ED8AA4A, 0x5B9CCA4F, 0x682E6FF3,
0x748F82EE, 0x78A5636F, 0x84C8B714, 0x8CC70208,
0x90BEFFFA, 0xA4506CEB, 0xBEF9A3F7, 0xC67178F2
};

// Write the SHA256 H array
arraycheck("Write SHA256 H to memory", 0x40020000, sha256, 72);
singlecheck("HASH_crfunc ", 0x4002b000, 0xff); //Hash Init
singlecheck("HASH_ar ", 0x4002b004, 0x5a);
singlecheckwait("sfr done check", 0x4002b00c, 0x1);
singlecheck("fr write 1 ", 0x4002b00c, 0x1);

```

Before entering secure mode, all hash parameters should be set by CPU, it can be done by the old ways(software), the screenshot just initialized SHA256 parameter for HMAC256.

Step 4: Load key and secret

```

*****Write data to memory using CPU*****
singlecheck("GLB_suben ", 0x0020904, 0x1f); //sub clik enable

uint32_t hmac_key1[] = {0x0010203, 0x4050607, 0x8090a0b, 0xc0d0e0f, 0x10111213, 0x14151617, 0x18191a1b, 0x1c1d1e1f};
uint32_t hmac_key2[] = {0x0010203, 0x4050607, 0x8090a0b, 0xc0d0e0f, 0x10111213, 0x14151617, 0x18191a1b, 0x1c1d1e1f};
uint32_t hmac_key3[] = {0x0010203, 0x4050607, 0x8090a0b, 0xc0d0e0f, 0x10111213, 0x14151617, 0x18191a1b, 0x1c1d1e1f};
uint32_t hmac_key4[] = {0x0010203, 0x4050607, 0x8090a0b, 0xc0d0e0f, 0x10111213, 0x14151617, 0x18191a1b, 0x1c1d1e1f};

uint32_t hmac_key_padding[] = { 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0 };

uint32_t hmac_secret1[] = {0xe9499be4, 0xf298d408, 0xf13222a4, 0x18d1f3b6, 0x97d53fdd, 0xf590c210, 0x0c120400, 0x09b70711};
uint32_t hmac_secret2[] = {0x178ee48a, 0xb27c7f70, 0x8cc07fb7, 0x2baa20e3, 0x0e927f10, 0x1cc20da3, 0x4090400f, 0x336d2900};
uint32_t hmac_secret3[] = {0x29f97748, 0x1c8074da, 0xa703f038, 0x68ca2fbc, 0xc0ce0009, 0x215321c7, 0xb0a87130, 0x30a0e08c};
uint32_t hmac_secret4[] = {0xc71d001c, 0x57bd1500, 0x0859c150, 0x9704c3e0, 0xc60f049, 0x090f0eb9, 0xaf112b3b, 0x4f225007};

uint32_t hmac_msg[] = {0x10203, 0x4050607, 0x0090a0b, 0xc0d0e0f, 0x10111213, 0x14151617, 0x18191a1b, 0x1c1d1e1f, 0x00000000, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x00};

arraycheck_reram("\nWrite HMAC256 secret 1 to ReRAM data slot through CPU", 0x603f0020, hmac_secret1, 0);
//arraycheckread("Array check read testing, reading from", 0x603f0000, 1);

arraycheck_reram("\nWrite HMAC256 key 1 to ReRAM through CPU", 0x603f0000, hmac_key1, 0);
arraycheckread("Array check read testing, reading from", 0x603f0020, 1);

arraycheck_reram("\nWrite HMAC256 secret 2 to ReRAM data slot through CPU", 0x603f0000, hmac_secret2, 0);
//arraycheckread("Array check read testing, reading from", 0x603f0000, 1);

arraycheck_reram("\nWrite HMAC256 key 2 to ReRAM through CPU", 0x603f0000, hmac_key2, 0);
//arraycheckread("Array check read testing, reading from", 0x603f0000, 1);

arraycheck_reram("\nWrite HMAC256 secret 3 to ReRAM data slot through CPU", 0x603f0000, hmac_secret3, 0);
//arraycheckread("Array check read testing, reading from", 0x603f0000, 1);

arraycheck_reram("\nWrite HMAC256 key 3 to ReRAM through CPU", 0x603f0000, hmac_key3, 0);
//arraycheckread("Array check read testing, reading from", 0x603f0000, 1);

arraycheck_reram("\nWrite HMAC256 secret 4 to ReRAM data slot through CPU", 0x603f0000, hmac_secret4, 0);
//arraycheckread("Array check read testing, reading from", 0x603f0000, 1);

arraycheck_reram("\nWrite HMAC256 key 4 to ReRAM through CPU", 0x603f0000, hmac_key4, 0);
//arraycheckread("Array check read testing, reading from", 0x603f0000, 1);

arraycheck_reram("\nWrite HMAC256 key zero padding to ReRAM data slot through CPU", 0x603e0000, hmac_key_padding, 0);
//arraycheckread("Array check read testing, reading from ", 0x603e0000, 0);

arraycheck("Write HMAC256 message to SRAM through sdr", 0x0110000c, hmac_msg, 10);
//arraycheckread("Array check read testing, reading from 0x0110000c", 0x0110000c, 4);

```

Load secret to the odd number slots and corresponding key to even number slots, I set the message be the same for all key & secret sets, it can be set different.

Key (hex): 000102030405060708090A0B0C0D0E0F101112131415161718191A1B1C1D1E1F
 Message (hex): 000102030405060708090A0B0C0D0E0F101112131415161718191A1B1C1D1E1F
 HMAC-SHA256 (hex): e8499be4f1980d68f13222a418df5cbd97d53fddf590c2108e22d40005b70713

Key (hex): 020102030405060708090A0B0C0D0E0F101112131415161718191A1B1C1D1E1F
 Message (hex): 000102030405060708090A0B0C0D0E0F101112131415161718191A1B1C1D1E1F
 HMAC-SHA256 (hex): 1f8ee48ab2fcff568cc87fb72baa2e638e92ff101cc2eda5409040bf536d29b6

Key (hex): 030102030405060708090A0B0C0D0E0F101112131415161718191A1B1C1D1E1F
 Message (hex): 000102030405060708090A0B0C0D0E0F101112131415161718191A1B1C1D1E1F
 HMAC-SHA256 (hex): 29f977e81c8074daa703f0d848c42fbc1dce6eb9215321c7bba8725b300a08ca

Key (hex): 040102030405060708090A0B0C0D0E0F101112131415161718191A1B1C1D1E1F
 Message (hex): 000102030405060708090A0B0C0D0E0F101112131415161718191A1B1C1D1E1F
 HMAC-SHA256 (hex): c71dba1c87bd1566d859c15a97b4c5dec6e8fd490d0d1eb9af312b5b4f225087

Step 5: Enter Secure mode and Transfer data

```

/*****Entering secure mode*****/
singlecheck_print("Entering secure mode", 0x40028000, 0x2); // secure mode, enter secure mode before accessing key slot
*((volatile uint16_t*) 0x40000000) = 0x0c00; // Error ER bit

for (int i = 0; i < 4; i++){
/*****Loading data from ReRAM to SCERAM*****/
fprintf(stderr, "HMAC key number: %d\n", i+1);
uint32_t base_key_addr = 0x603f0000; // start from the first key
//uint32_t base_secret_addr = 0x603f0000; // start from the first secret

singlecheck("SDMA_schcr_opt ", 0x40029034, 0x100); // secure + privileged mode
singlecheck("SDMA_schcr_func ", 0x40029030, 0x0); // choose 0 read or 1 write
singlecheck_print("SDMA_schcr_axstart ", 0x40029038, base_key_addr + i*0x40 ); // AXI start address, read from long key, the address should be the destination address
singlecheck("SDMA_schcr_segid ", 0x4002903c, 0x01); // Indicate the segment you want to write
singlecheck("SDMA_schcr_transize ", 0x40029044, 8); // transfer size from to ReRAM must be 256 bits
singlecheck("SDMA_schcr_segstart ", 0x40029040, 0x0); //segment pointer
singlecheck("SDMA_chstart_ar ", 0x40029000, 0xaa); //channel start
singlecheckwait("Loading key to SCERAM, sfr done check", 0x40028014, 0x40 ); // waiting for sch done flag
//arraycheckread("ReRAM write to SCERAM, Reading Key from KEY", 0x40020100, 1);

singlecheck("SDMA_schcr_opt ", 0x40029034, 0x100); // secure + privileged mode
singlecheck("SDMA_schcr_func ", 0x40029030, 0x0); // choose 0 read or 1 write
singlecheck("SDMA_schcr_axstart ", 0x40029038, 0x611000cc); // AXI start address, read from long key, the address should be the destination address
singlecheck("SDMA_schcr_segid ", 0x4002903c, 0x04); // Indicate the segment you want to write
singlecheck("SDMA_schcr_transize ", 0x40029044, 32); // how many word you want to move
singlecheck("SDMA_schcr_segstart ", 0x40029040, 0x0); //segment pointer
singlecheck("SDMA_chstart_ar ", 0x40029000, 0xaa); //channel start
singlecheckwait("Loading MSG to SCERAM, sfr done check", 0x40028014, 0x40 ); // waiting for sch done flag
//arraycheckread("SRAM write to SCERAM, Reading Key from MSG", 0x40020400, 1);

singlecheck("SDMA_schcr_opt ", 0x40029034, 0x100); // secure + privileged mode
singlecheck("SDMA_schcr_func ", 0x40029030, 0x0); // choose 0 read or 1 write
singlecheck("SDMA_schcr_axstart ", 0x40029038, 0x603e0000); // AXI start address, read from long key, the address should be the destination address
singlecheck("SDMA_schcr_segid ", 0x4002903c, 0x01); // Indicate the segment you want to write
singlecheck("SDMA_schcr_transize ", 0x40029044, 8); //how many word you want to move
singlecheck("SDMA_schcr_segstart ", 0x40029040, 0x0); //segment pointer
singlecheck("SDMA_chstart_ar ", 0x40029000, 0xaa); //channel start
singlecheckwait("Loading key to SCERAM, sfr done check", 0x40028014, 0x40 ); // waiting for sch done flag
//arraycheckread("ReRAM write to SCERAM, Reading Key from KEY", 0x40020100, 1);

```

We need to make SCE to secure mode by writing 0x2 to sfr_scemode
 Then we can start SDMA to load data from ReRAM to SCE segment
 singlecheck ("SDMA_schcr_opt ", 0x40029034, 0x100);

We need to set the axi_prot to secure access, so bit [9] is 0, prrot is privilege mode, [8] is set 1.
 singlecheck("SDMA_schcr_func ", 0x40029030, 0x0); // choose 0 read or 1 write
 singlecheck_print("SDMA_schcr_axstart ", 0x40029038, base_key_addr + i*0x40);
 // AXI start address, read from long key, the address should be the destination address

We are reading from key slot 0, put its address here
 singlecheck("SDMA_schcr_segid ", 0x4002903c, 0x01);
 // Indicate the segment you want to write first we write to SEG_KEY, segid is 1

 singlecheck("SDMA_schcr_transize ", 0x40029044, 8); // transfer size from to ReRAM must be 256 bits
 singlecheck("SDMA_schcr_segstart ", 0x40029040, 0x0); //segment pointer
 singlecheck("SDMA_chstart_ar ", 0x40029000, 0xaa); //channel start
 singlecheckwait("Loading key to SCERAM, sfr done check", 0x40028014, 0x40); // sch done flag

Step 6: Turn on HMAC

Turn on trust state mode and secret checking by setting opt2 to 0x25, the HMAC will compare the result with the secret stored in ReRAM key slot. If they are equal, a truststate flag will be raised.

```

/*****Turn on HMAC*****/

singlecheck("HASH_crfunc ", 0x4002b000, 0x50); // pass 1
singlecheck("HASH_opt1 ", 0x4002b010, 0x0); //
singlecheck("HASH_opt2 ", 0x4002b014, 0x4); //
singlecheck("HASH_opt3 ", 0x4002b018, 0x0); //
singlecheck("HASH_ar ", 0x4002b004, 0x5a);
singlecheckwait("HMAC step1, sfr done check", 0x4002b00c, 0x1 ); // waiting for sch done flag
// arraycheckread("SRAM write to SCERAM, Reading Result from HOUT", 0x40020600, 4);

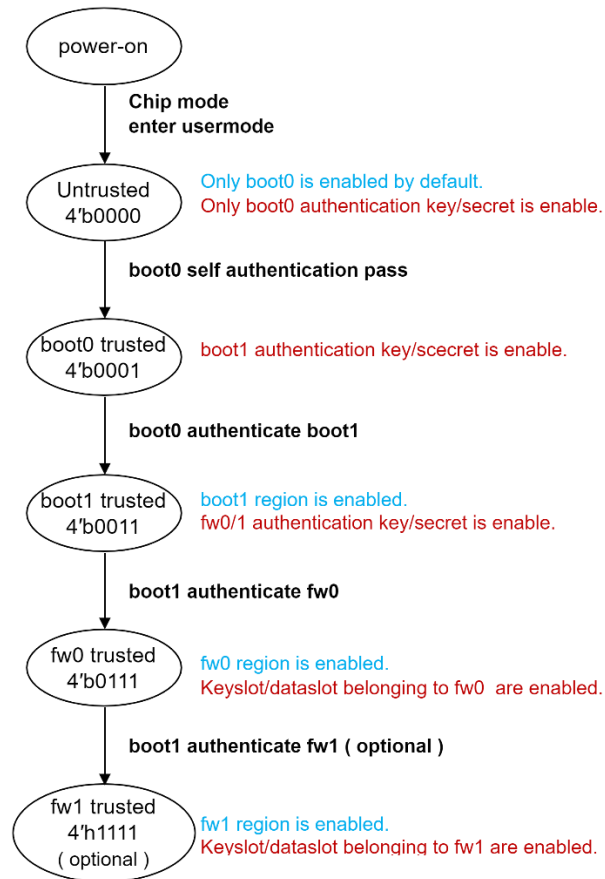
singlecheck_print("keyid ", 0x4002b060, i*2 +1); // check keyslot 1, 3, 5, 7
singlecheck("HASH_crfunc ", 0x4002b000, 0x60); // pass 2
singlecheck("HASH_opt1 ", 0x4002b010, 0x0); //
singlecheck_print("HASH_opt2 ts mode & check secret", 0x4002b014, 0x25); // ts mode & check secret
singlecheck("HASH_opt3 ", 0x4002b018, 0x0); //
singlecheck("HASH_ar ", 0x4002b004, 0x5a);
singlecheckwait("HMAC step2, sfr done check", 0x4002b00c, 0x1 ); // waiting for sch done flag
singlecheckread("HASH_fr secret check flag ", 0x4002b00c);

```

High-level secure bootup flow

The diagram below shows how the basic HMAC secure certificate applied in the high-level secure bootup sequence flow. Without this kind of flow, the boot1, fw0 and fw1 cannot execute any code in The DARIC.

- This authentication flow is used to check the code integrity. The following picture is just an example here to show how it work. It all depends on how the SW defines.
- The 4'bxxxx is used to stand for the boot0, boot1, fw0, fw1 authentication status, not any internal signals in the design. Just for your reference.
- Take one example here, you can read all the boot0 code and check its hash value to do the authentication flow. Or you can use the boot0 key to do any other kind of calculation to get the value to compare with the preset target value.
- If the results compare the same, then it shows the code stored in the ReRAM is correct and never changed. If they are different, it means the code is already changed by some invalid access or attack.



HMAC certificate secure bootup sequence diagram

2 *Design Review Information*

Topic of Review:

Date:

Related Documents:

Meeting Moderator:

Meeting Scribe:

Invitees	Attended (Y/N)	Invitees	Attended (Y/N)

Agenda/Issues discussed:

Outcome/Summary:

ID	Item	Owner	Date Due	Status	Closed?
1					
2					
3					

Table 2-1 Actions Items